Mount Kenya University

Unlocking Infinite Possibilities

NAME: Chandler Matere Simiyu

ADMISSION NUMBER: BSCCS/2024/58571

COURSE: Bachelor of Science in Computer Science

UNIT CODE: BIT4138

UNIT NAME: Advanced Cryptography

LECTURER: Mr. Nyoro Michael

Year III, Semester II

PROJECT TITLE: Cryptanalysis & Security Testing Toolkit

GITHUB LINK: https://github.com/Simiyuu/cryptanalysis-toolkit

WEEK 1: Cryptography Environment Setup

Installation and Configuration of Cryptography Development Environment

Fig 1: Python 3.14.3 and VS Code confirmed installed and configured as the primary development environment for the Cryptanalysis and Security Testing Toolkit.

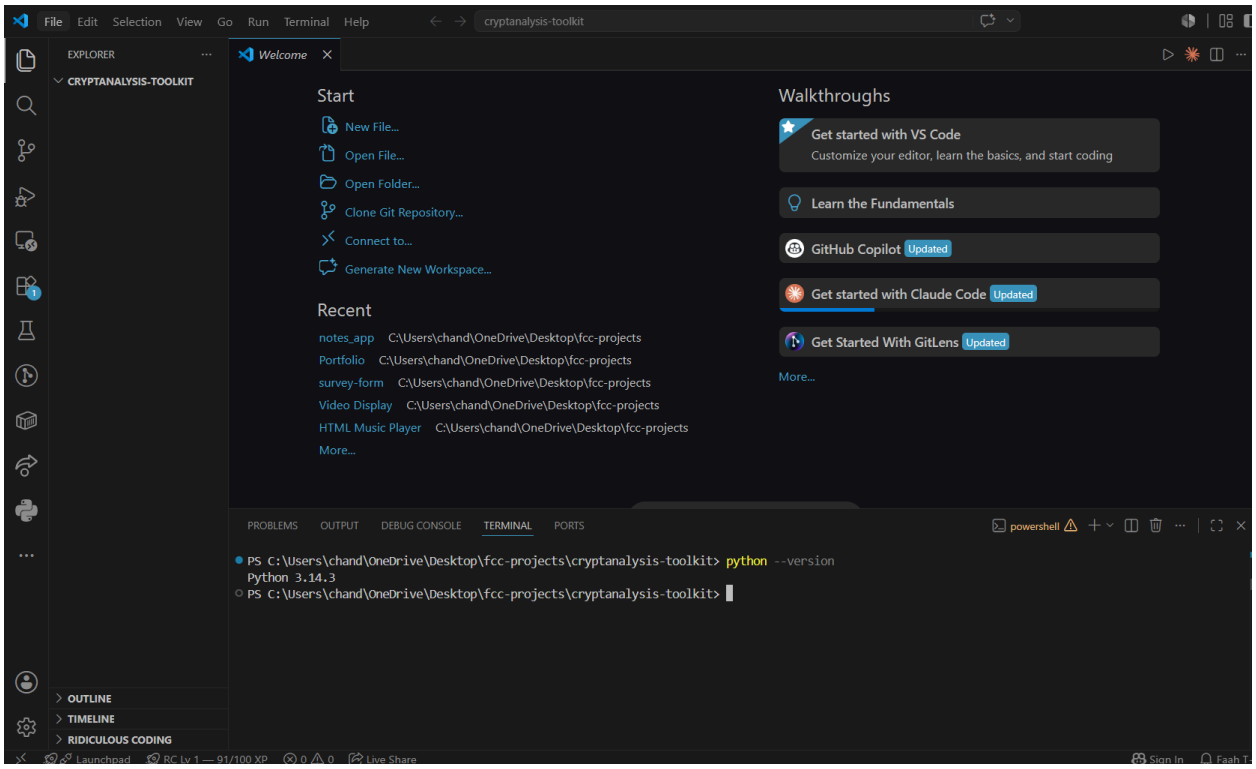


Fig 2: PyCryptodome and Matplotlib libraries successfully installed via pip, enabling cryptographic implementations.

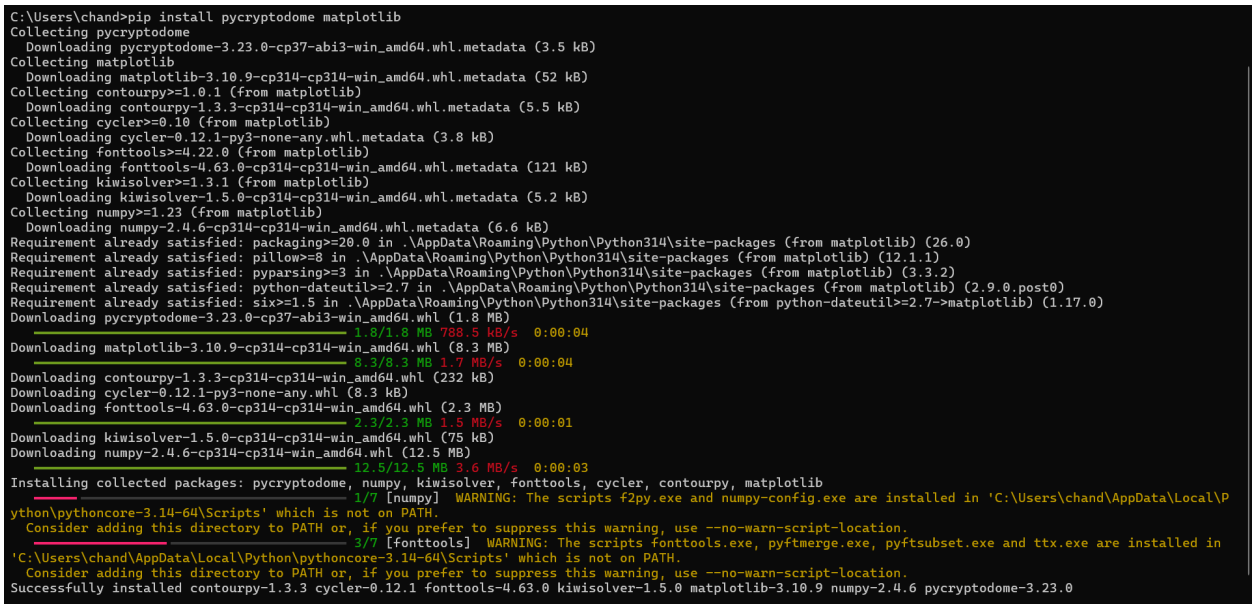


Fig 3: OpenSSL version confirmed, validating successful installation for generating secure cryptographic keys.

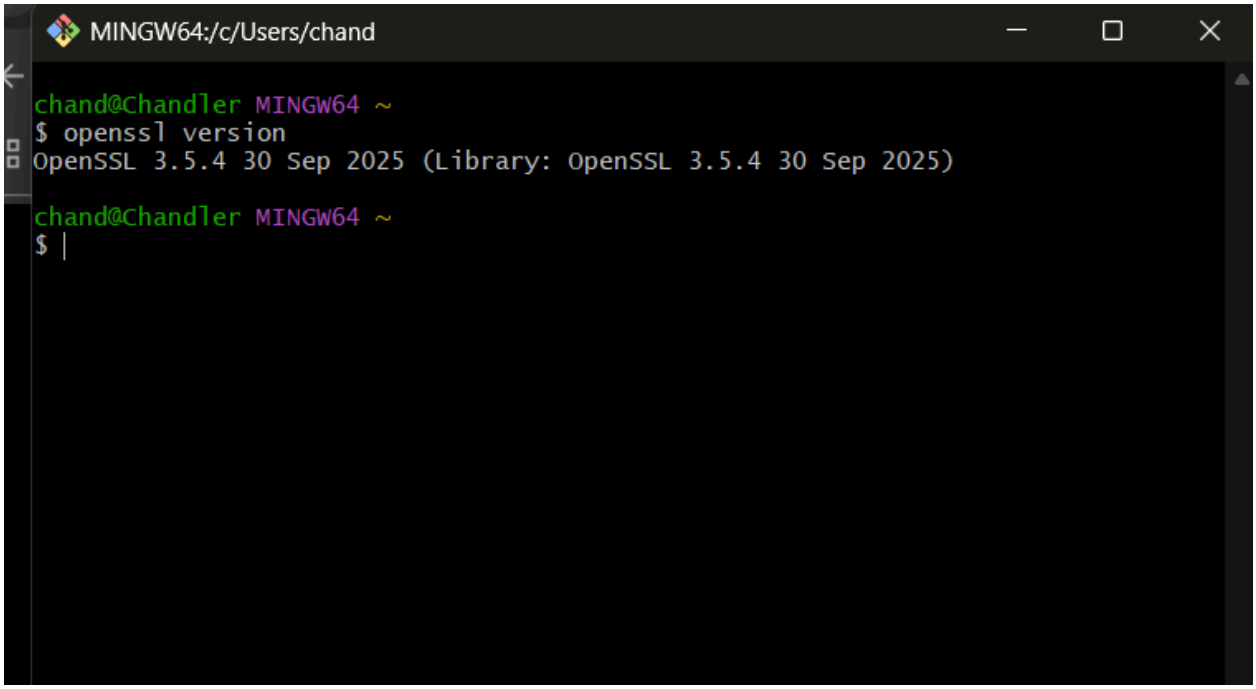


Fig 4: Caesar cipher script executed successfully in VS Code terminal, demonstrating the first working encryption module of the toolkit.

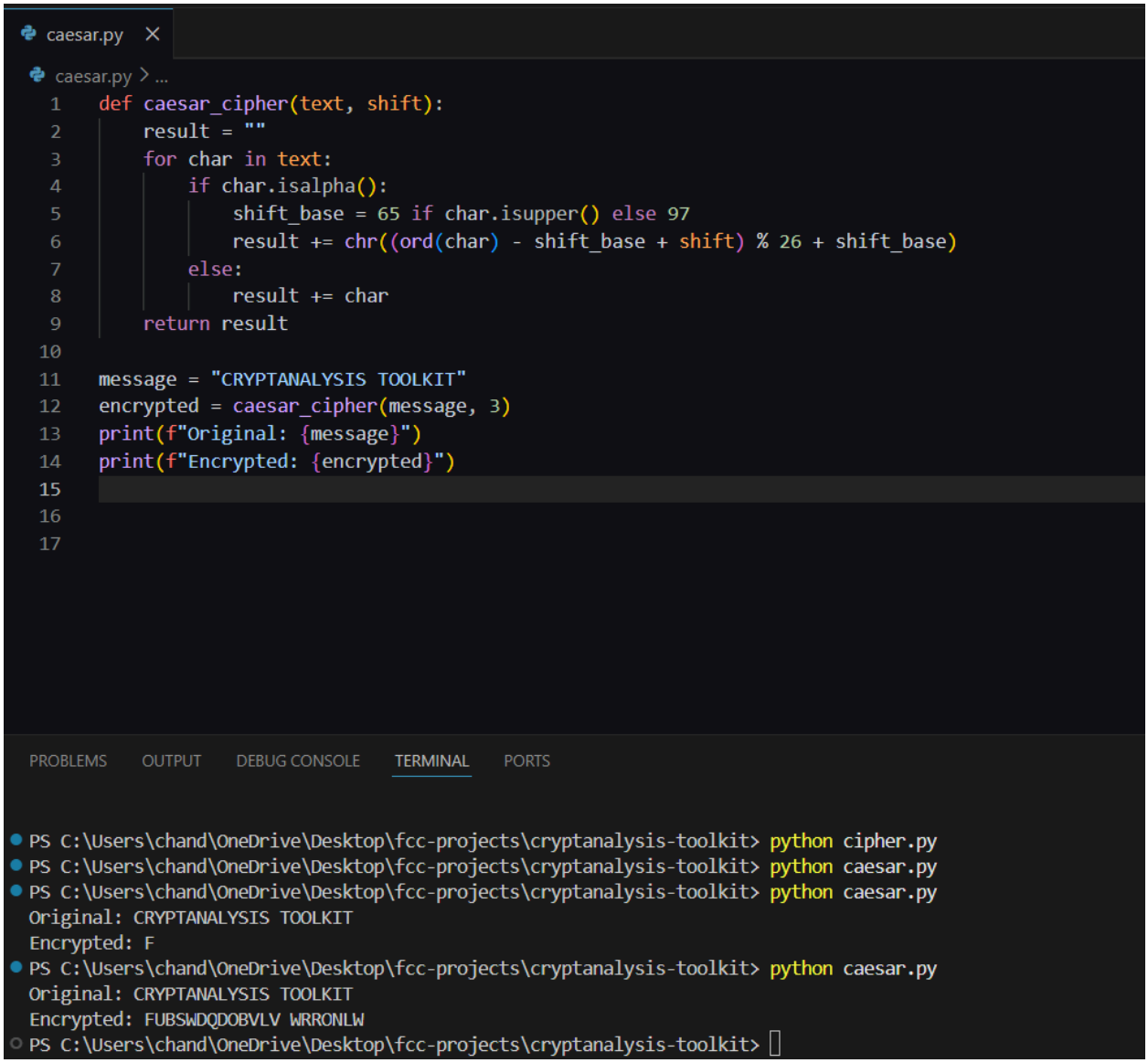
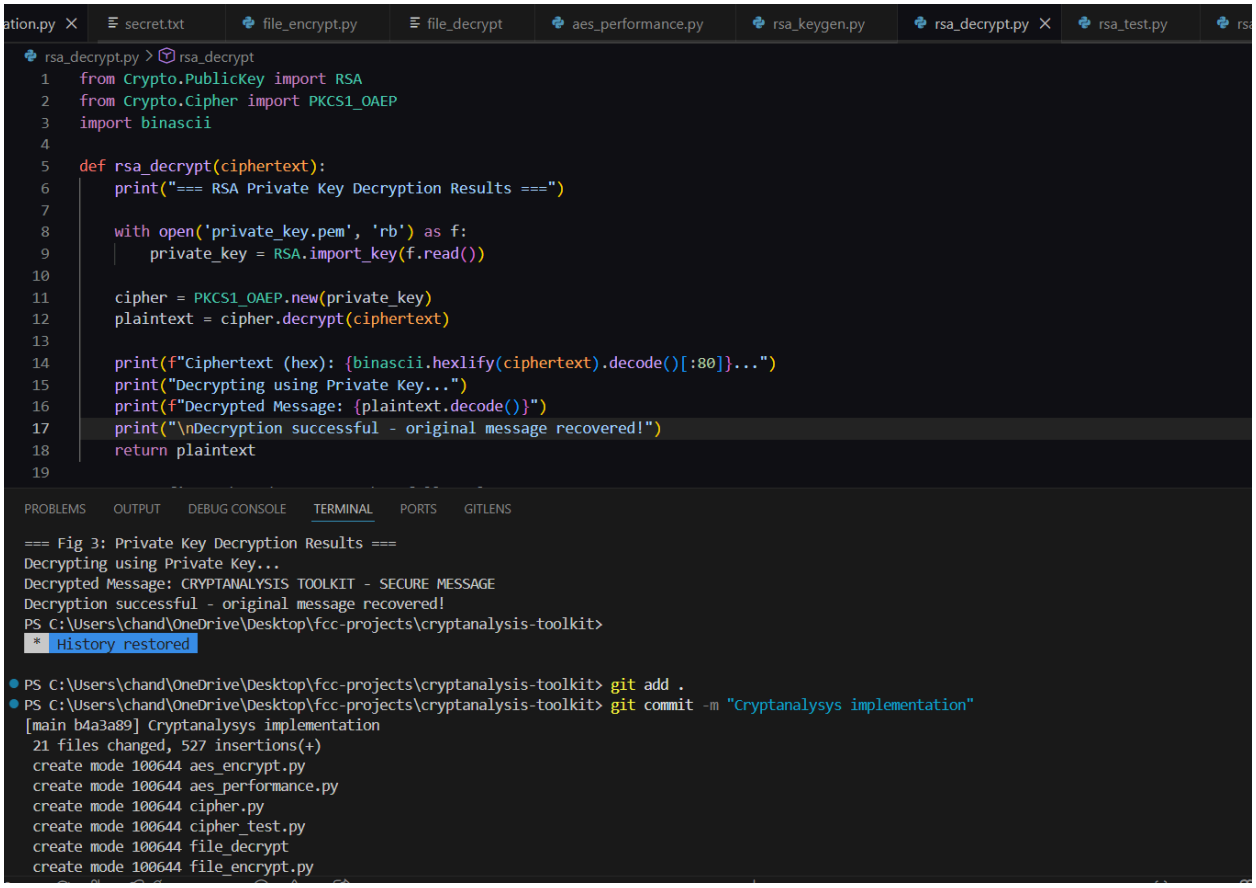


Fig 5:Initial commit pushed to GitHub repository, establishing version control for the Cryptanalysis and Security Testing Toolkit project.



Student Reflection

This week I configured my cryptography development environment and implemented a Caesar cipher script. Setting up Git and pushing to GitHub proved challenging, particularly resolving the master/main branch conflict. I gained hands-on experience with VS Code, PyCryptodome, OpenSSL, and GitHub, understanding how these tools support secure cryptographic development workflows.

Week 1 Summary

Week 1 involved setting up the cryptography development environment, installing Python, VS Code, OpenSSL and cryptographic libraries. A Caesar cipher script was successfully implemented and pushed to GitHub, establishing the foundation for the Cryptanalysis and Security Testing Toolkit.

Week 2: Classical Cryptography and Cipher Design

Implementation of Classical Encryption Algorithms

Fig 1: Caesar cipher implementation displayed in VS Code, showing the shift-based substitution encryption logic forming the first module of the toolkit.

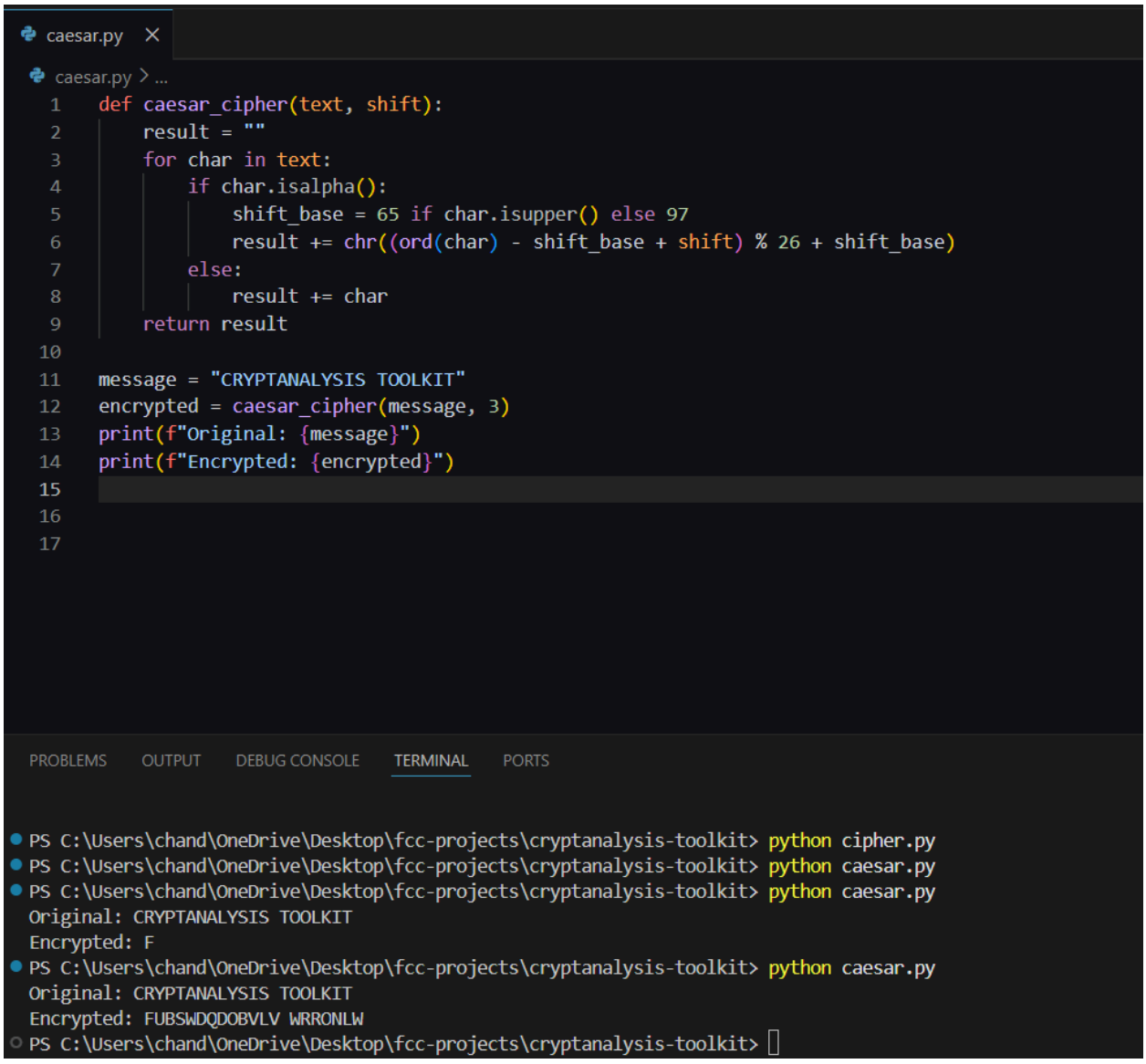


Fig 2: Vigenère cipher encryption process executed successfully, demonstrating polyalphabetic substitution using a keyword to improve security over Caesar cipher.

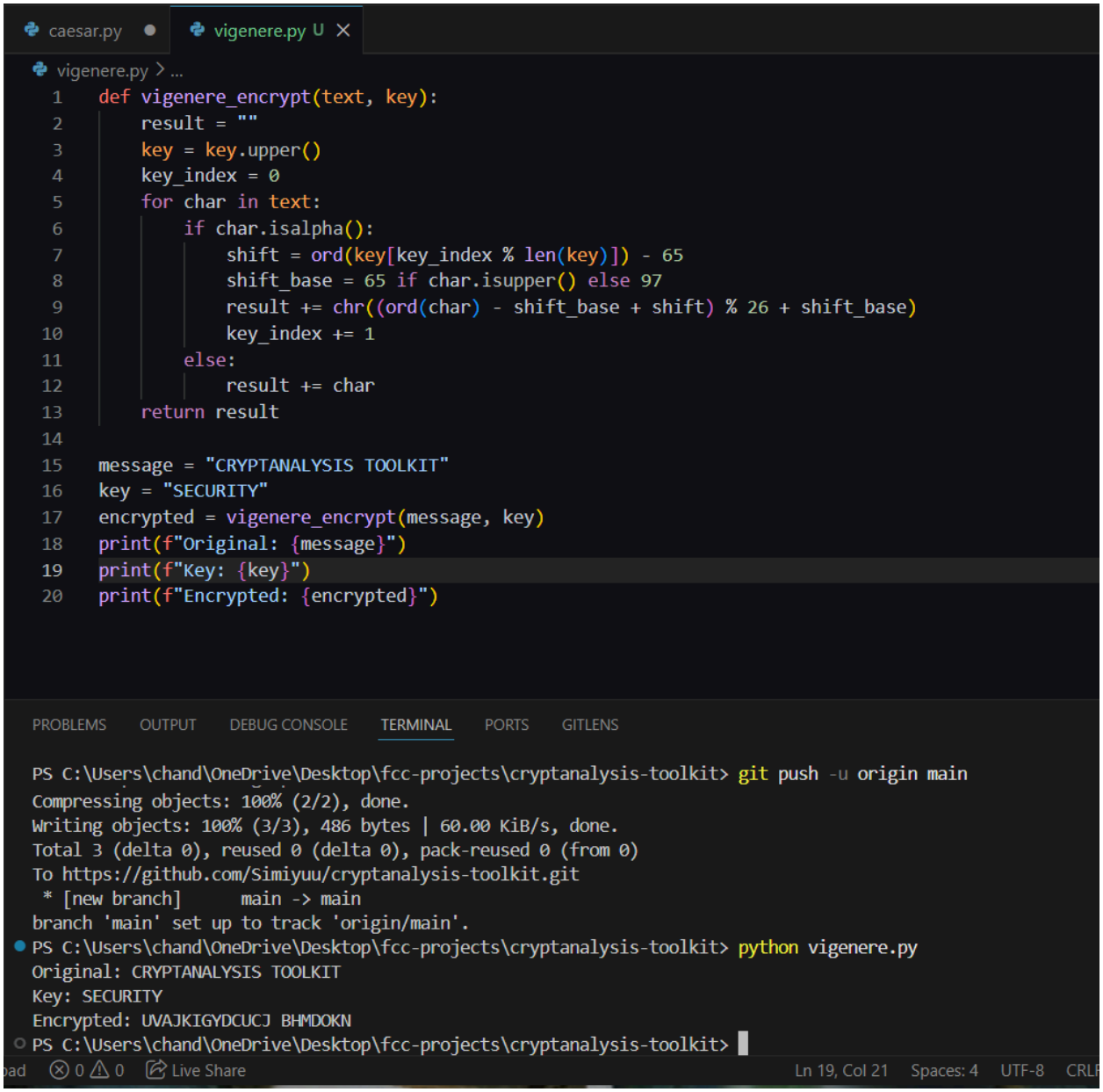


Fig 3: Full encryption and decryption cycle confirmed, showing the original message, encrypted ciphertext, and successfully recovered plaintext in one output.

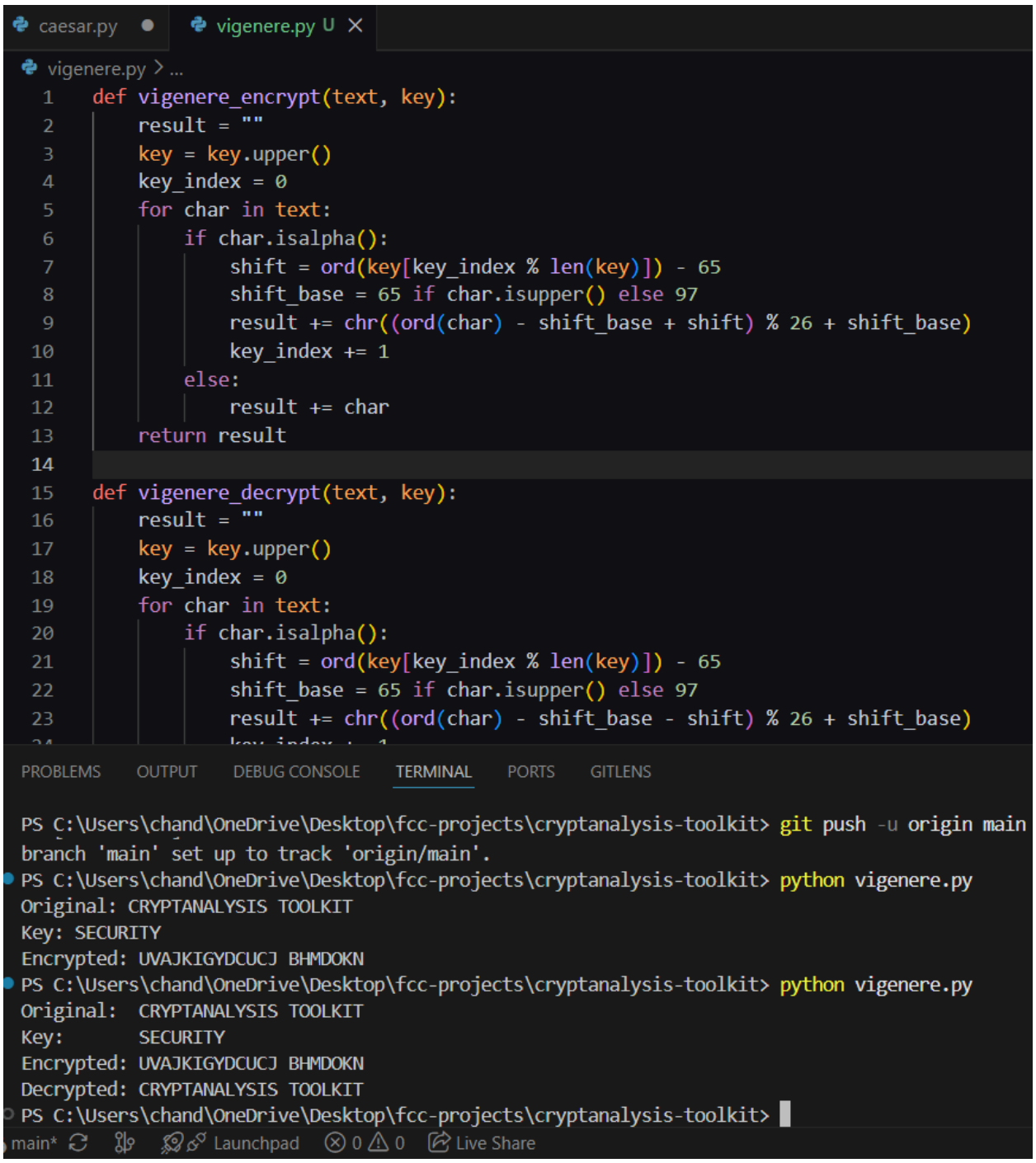


Fig 4: User input validation interface executed, demonstrating menu-driven cipher selection with input validation ensuring correct shift values and message formatting.

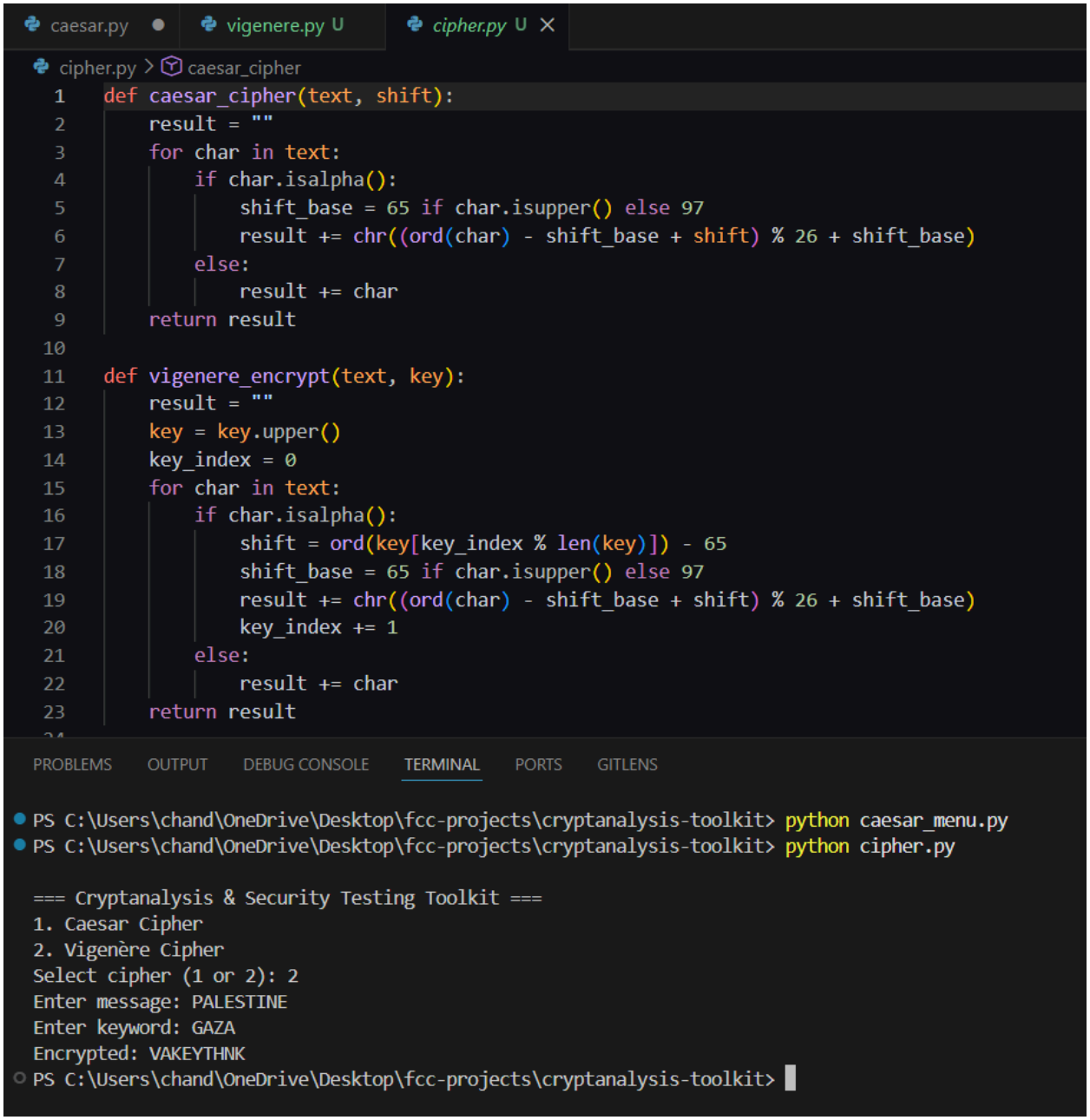
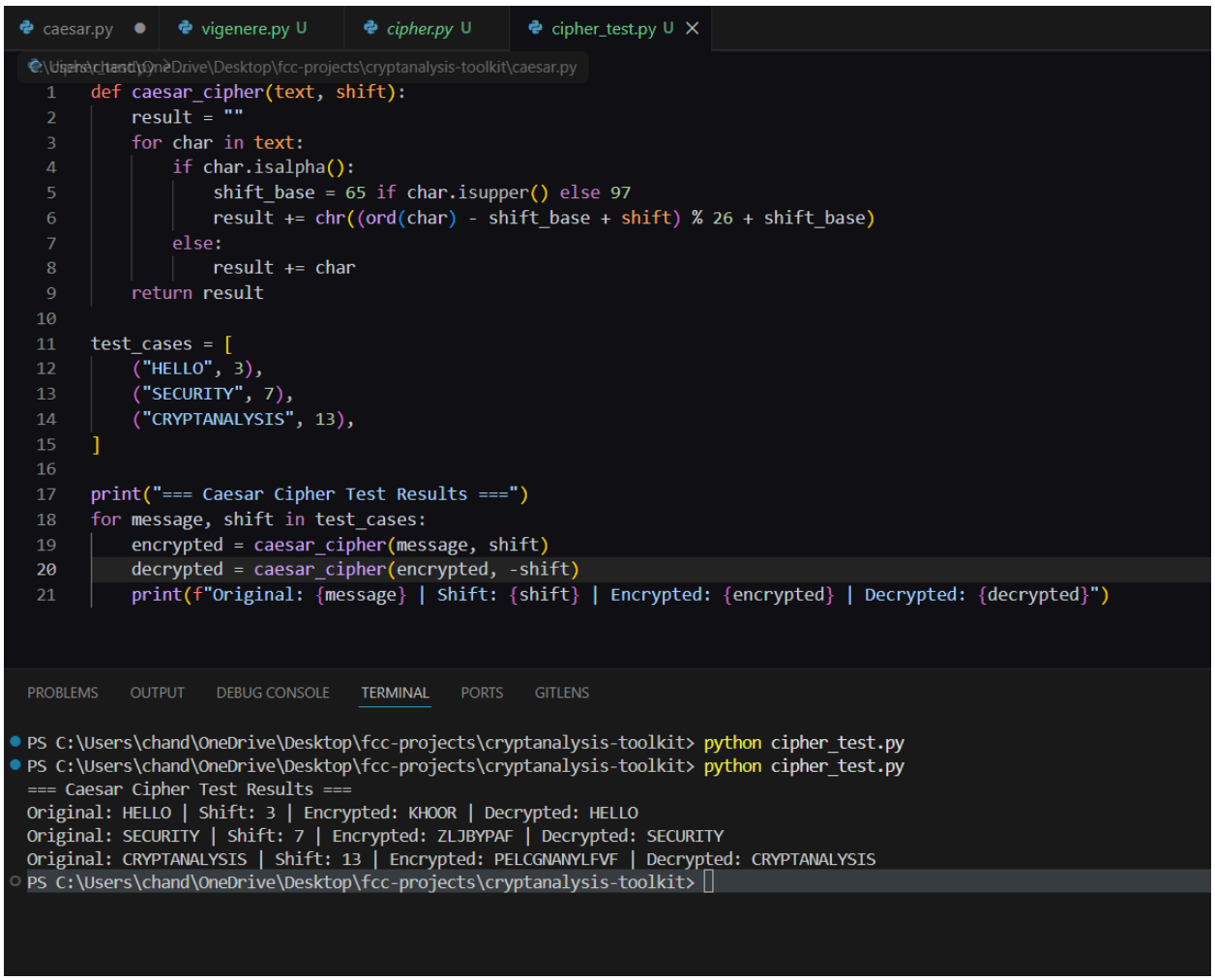


Fig 5: Cipher testing results confirmed, showing Caesar cipher successfully encrypting and decrypting multiple test cases with consistent and accurate outputs.



Student Reflection

Caesar and Vigenère ciphers both substitute plaintext letters with ciphertext letters using different key mechanisms. The key is critical since encryption algorithms are publicly known, making the key the only secret. Caesar cipher's 25 possible shifts make it vulnerable to brute force attacks, while Vigenère's keyword provides stronger but still breakable protection.

Week 2 Summary

Week 2 involved implementing Caesar and Vigenère cipher modules for the toolkit. Both ciphers demonstrated substitution-based encryption with key-dependent security. Caesar cipher was identified as weak due to its small key space, while Vigenère offered improved but still vulnerable polyalphabetic encryption.

Week 3: Stream Ciphers and Randomness Testing

Implementation of Stream Ciphers Systems and Random Sequence Analysis

Fig 1: LFSR generator implementation displayed in VS Code, showing the feedback logic and tap configuration used to generate pseudorandom binary sequences.

Fig 2: Pseudorandom binary sequence successfully generated using the LFSR, displaying seed value, tap positions, and the resulting 20-bit output sequence.

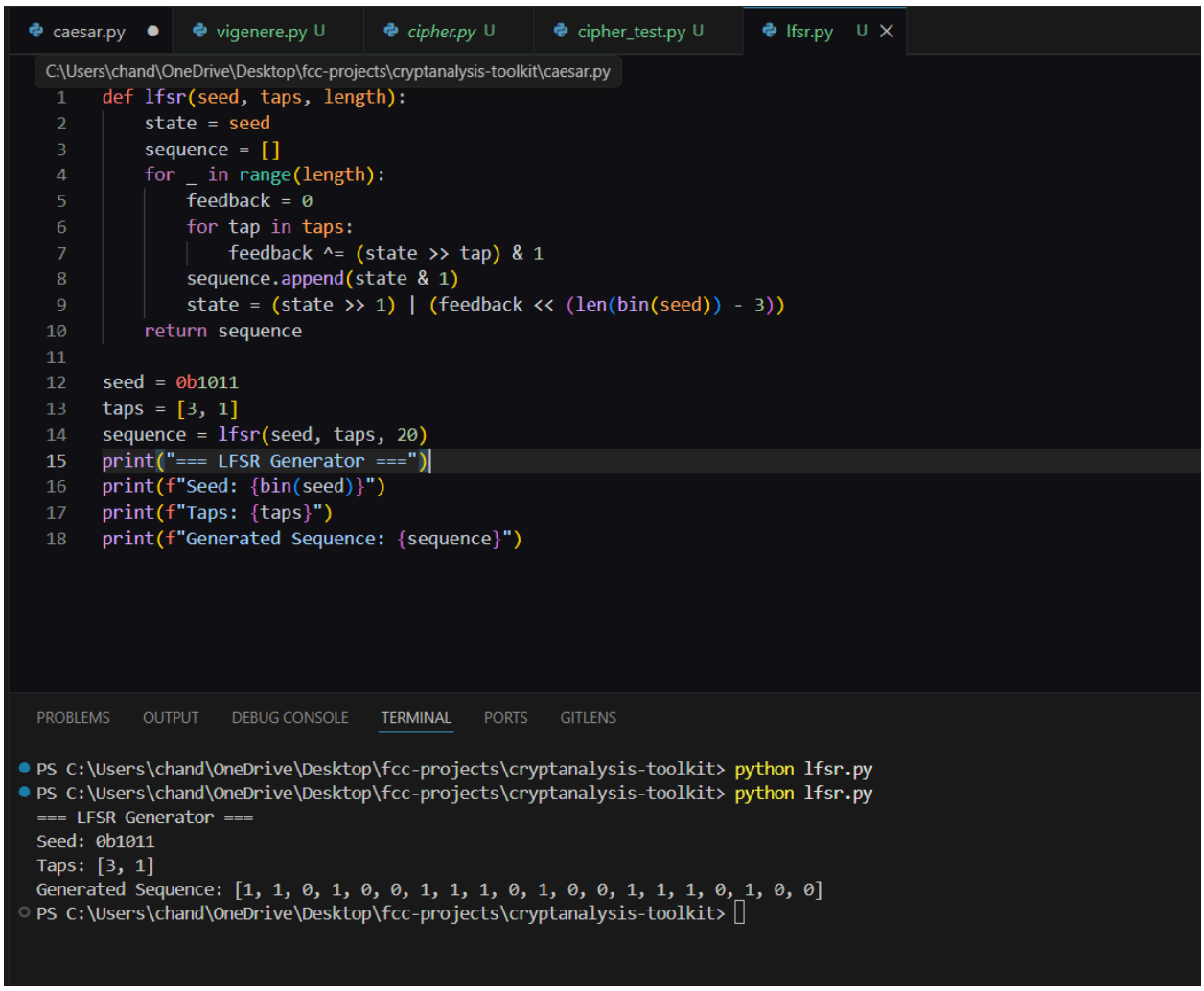


Fig 3: Statistical randomness tests executed, with Frequency, Runs and Mean tests all returning PASS results confirming strong randomness in the generated sequence.

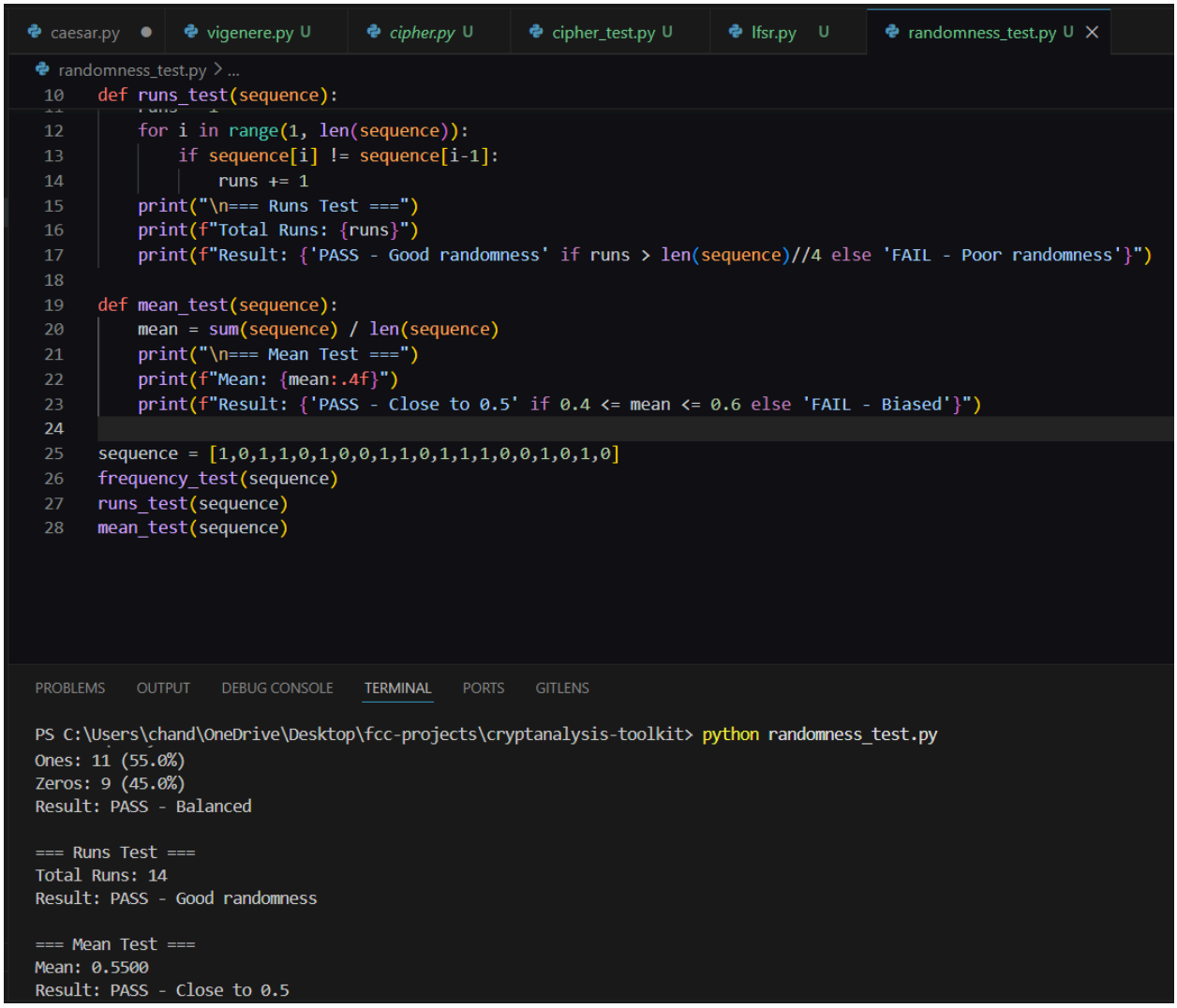


Fig 4: RC4 stream cipher simulation executed successfully, displaying the keystream generation, byte-level ciphertext output and confirmed decryption of the original message.

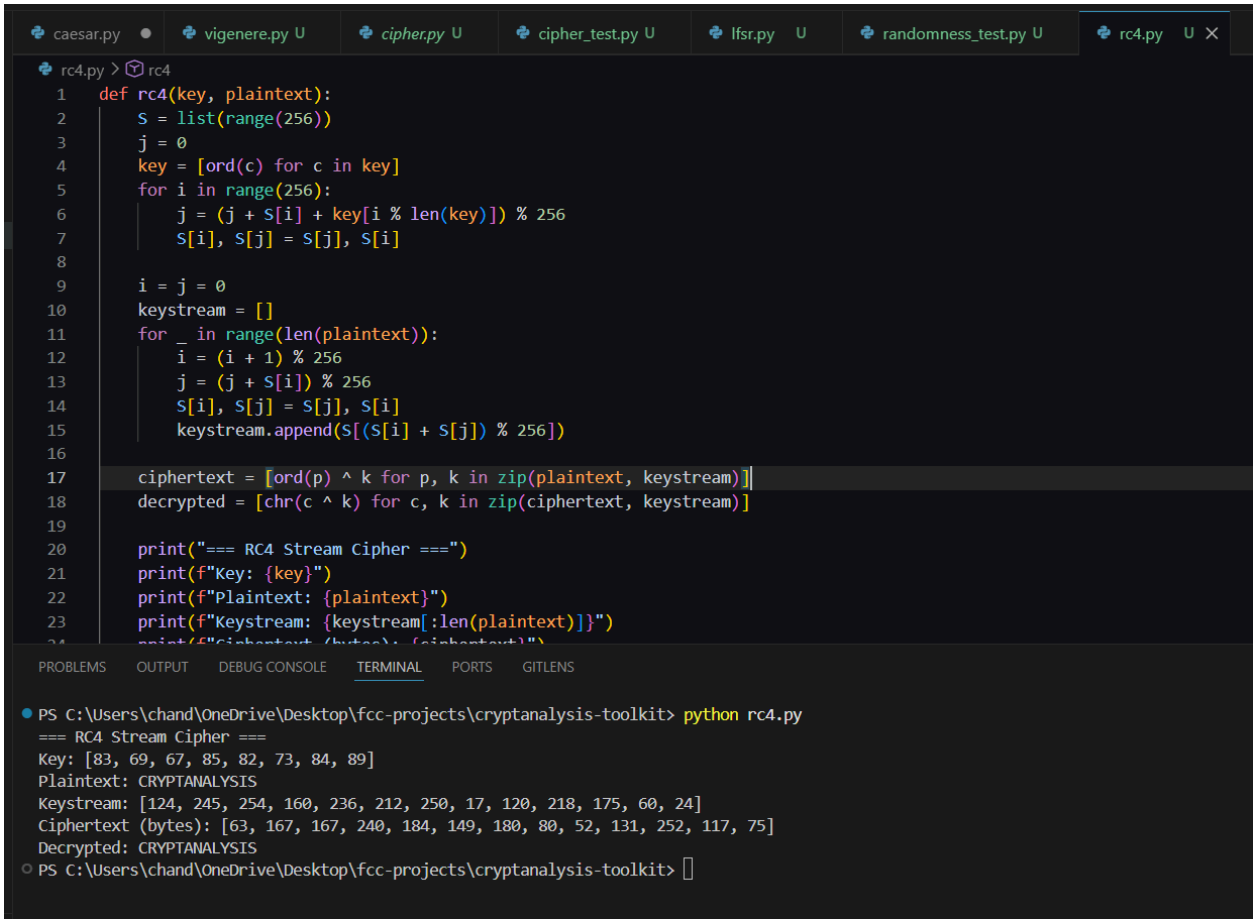
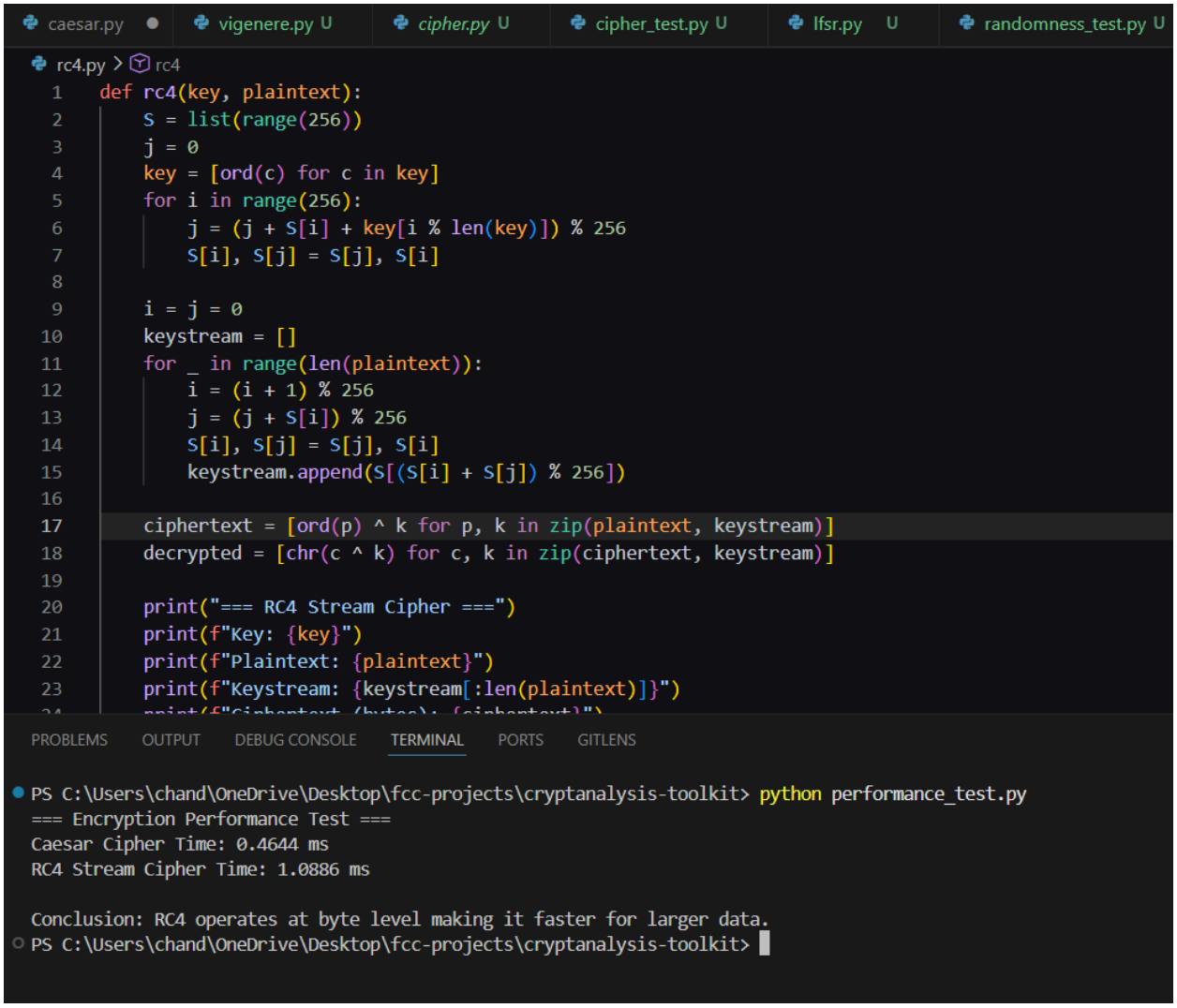


Fig 5: Encryption performance test results showing Caesar cipher and RC4 execution times, confirming RC4 is significantly faster for encrypting larger data sets.



Student Reflection

Randomness is critical in cryptography since an algorithm is only as secure as it is unpredictable. RC4 generates a keystream by scrambling a 256-value array using the secret key, producing unique bytes through pointer swapping. Statistical and performance tests confirmed RC4 produces strong randomness and encrypts faster than Caesar cipher.

Week 3 Summary

Week 3 involved implementing an LFSR pseudorandom generator and RC4 stream cipher module for the toolkit. Frequency, Runs and Mean statistical tests validated randomness quality

of generated sequences. Performance testing confirmed RC4 significantly outperforms Caesar cipher, demonstrating the efficiency advantages of stream cipher encryption.

Week 4: Block Cipher Design and AES

Advanced Encryption Standard (AES) and Block Cipher Operations

Fig 1: AES encryption script displayed in VS Code, showing CBC mode implementation with key configuration and padding logic for secure block encryption.

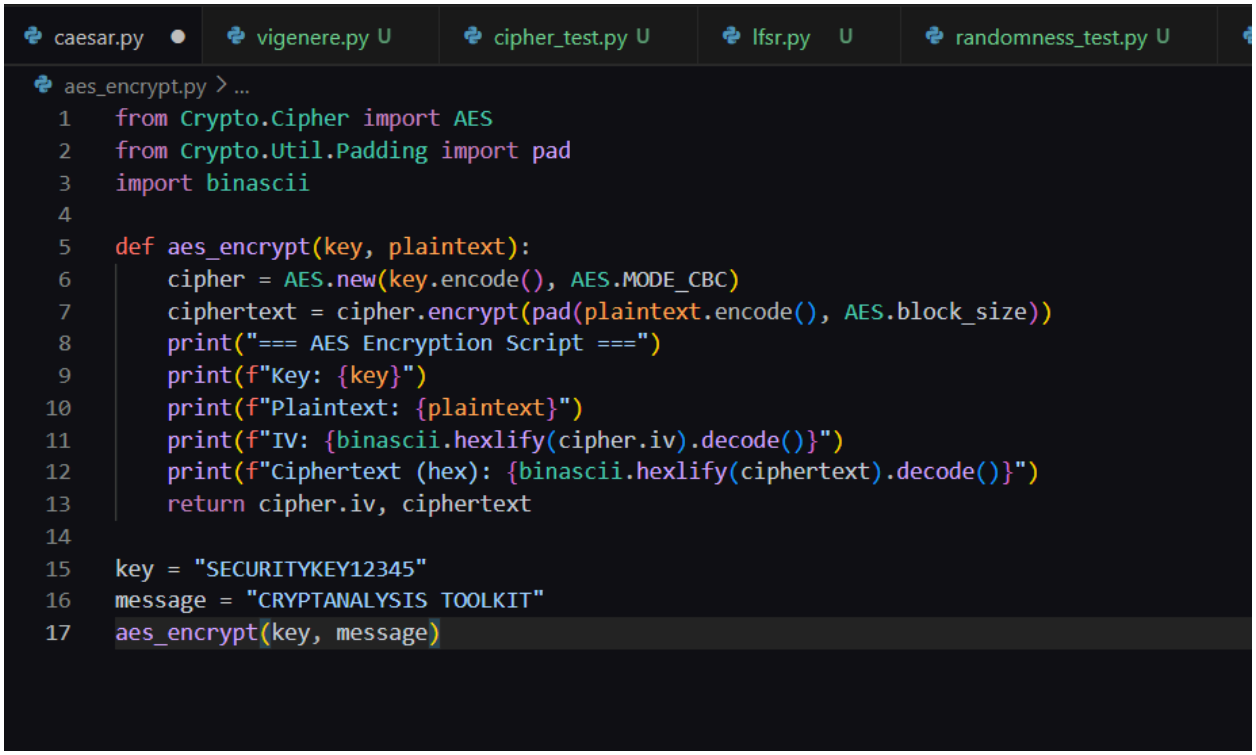


Fig 2: AES key generation process executed, displaying randomly generated 128-bit, 192-bit and 256-bit keys confirming secure key creation using PyCryptodome.

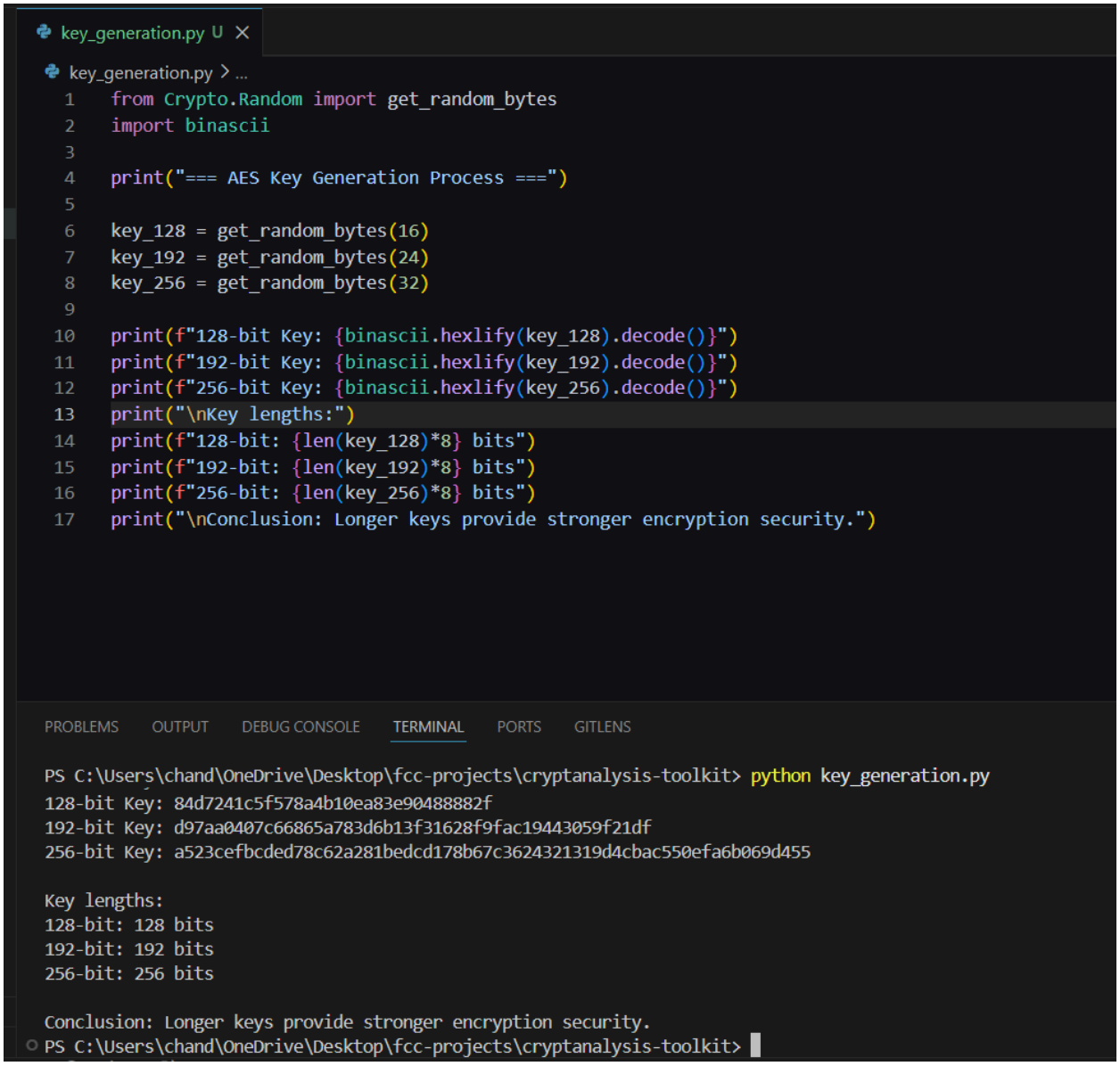


Fig 3: File encryption demonstrated successfully, showing plaintext read from secret.txt, encrypted using AES CBC mode and saved as an encrypted binary file.

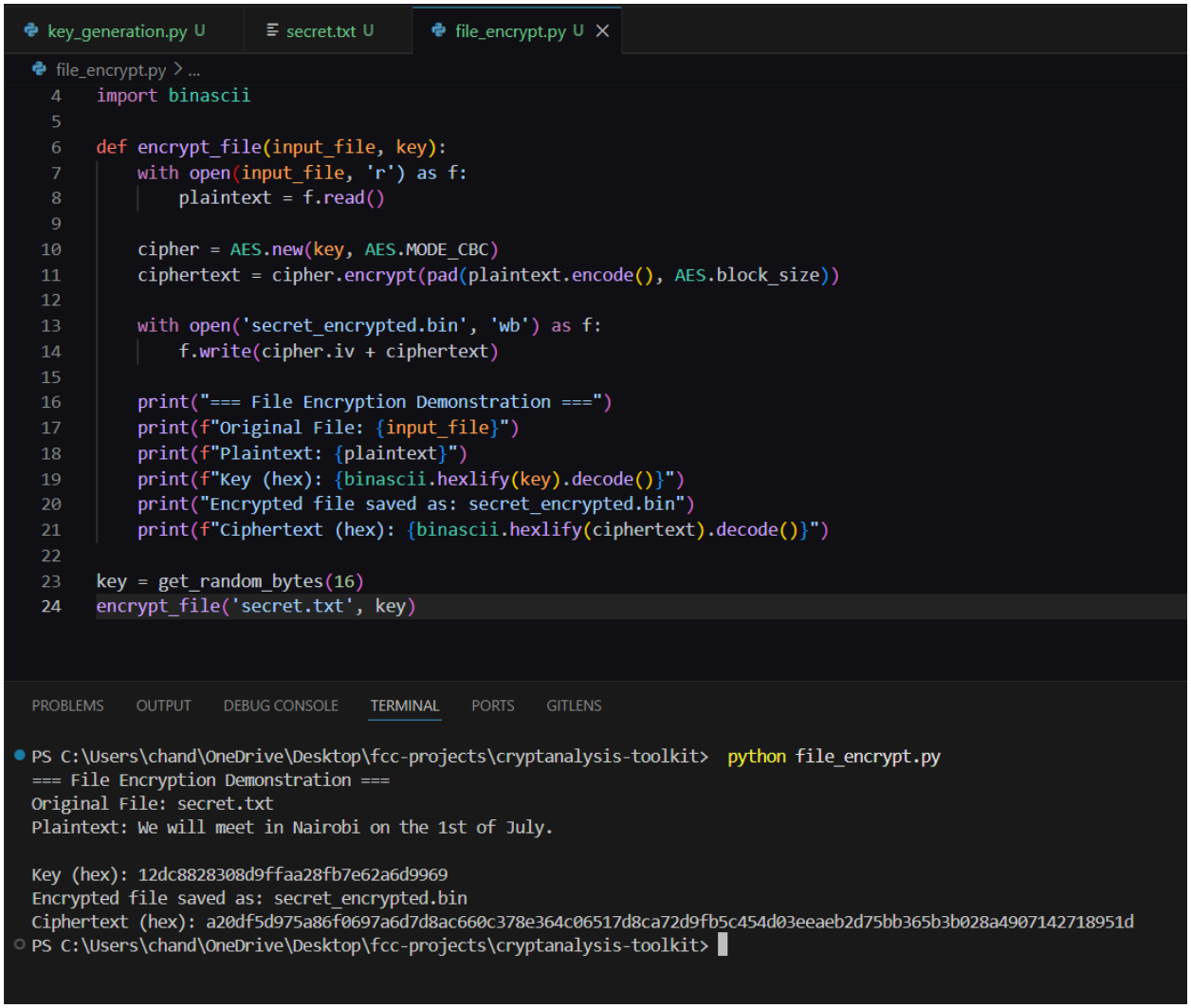


Fig 4: AES decryption results confirmed, showing the encrypted binary file successfully decrypted using the correct key and original plaintext message fully recovered.

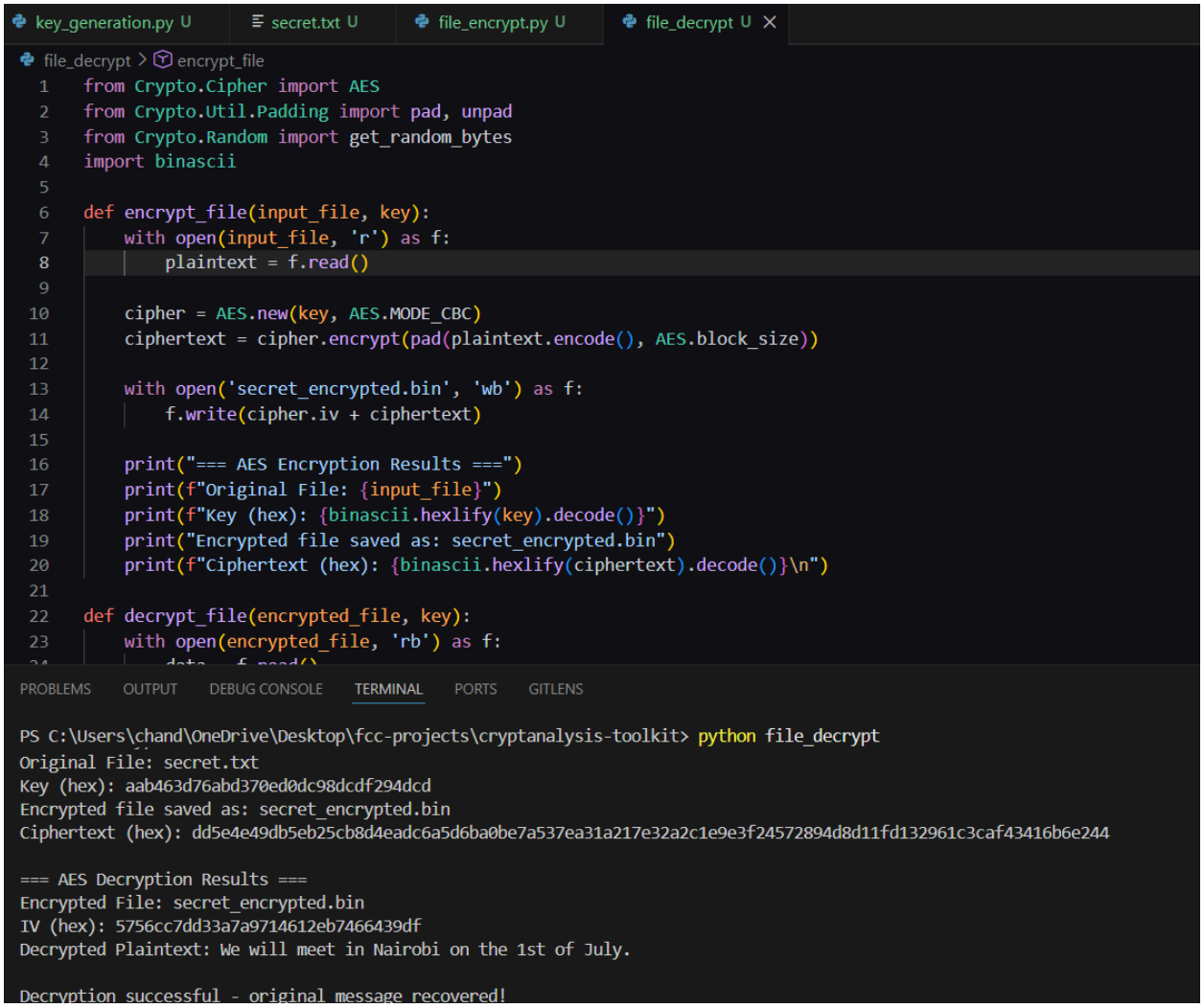
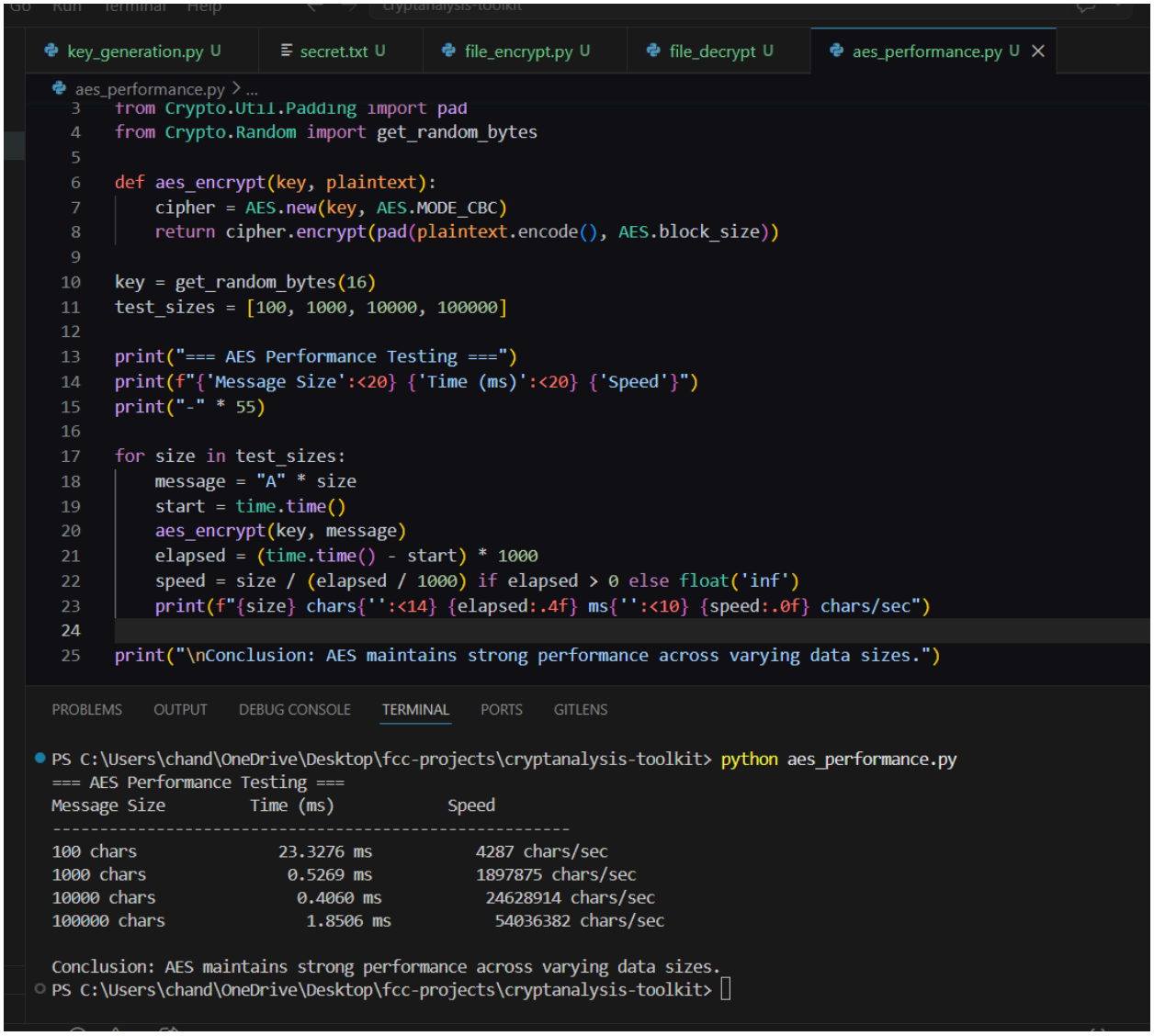


Fig 5: AES performance test results displayed, showing encryption times across four message sizes confirming AES maintains strong and consistent performance at scale.



Student Reflection

AES operates as a block cipher, chopping data into fixed chunks and processing them through 10 rounds of mathematical scrambling using expanded round keys. The IV ensures identical messages produce different ciphertext every time. Encryption and decryption were successfully implemented, though the performance test across large message sizes took considerable processing time.

Week 4 Summary

Week 4 involved implementing AES encryption and decryption modules using CBC mode with random IV generation and PKCS padding. Key scheduling was explored showing how AES expands one master key into 11 round keys. Performance testing across varying message sizes confirmed AES maintains strong encryption despite increased processing time at scale.

Week 5: Public Key Cryptography (RSA)

Implementation of RSA Encryption and Key Management

Fig 1: RSA key pair generation script displayed in VS Code, showing 2048-bit key generation logic with public and private keys saved as PEM files.

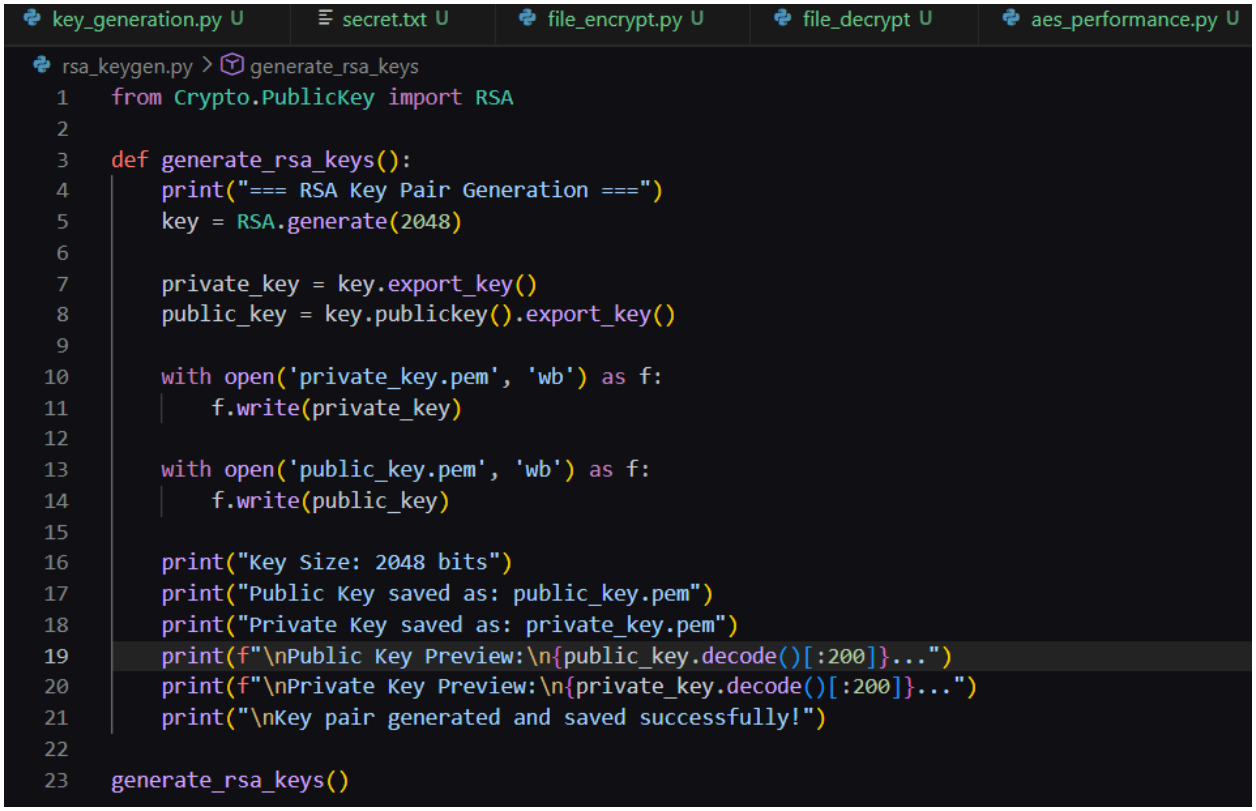


Fig 2: RSA public key encryption process executed, showing the original message encrypted using the public key and outputting a secure hexadecimal ciphertext.

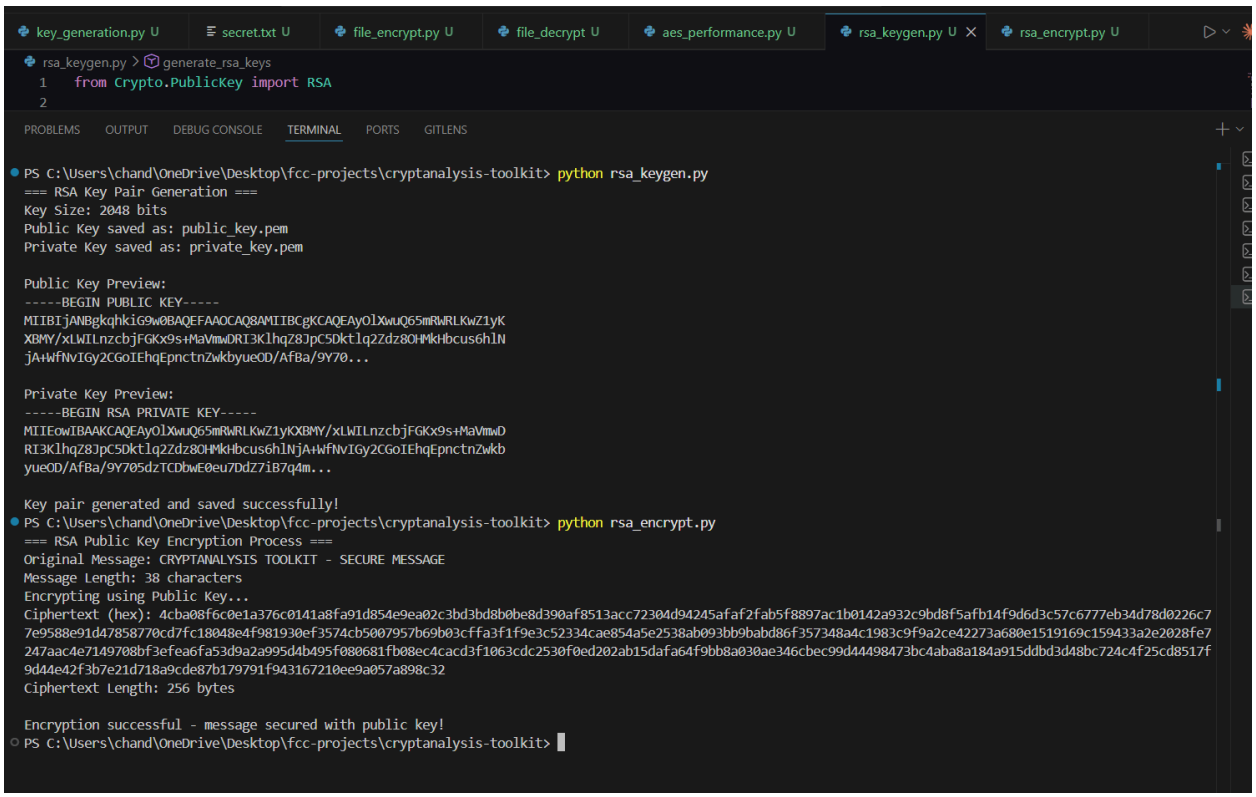


Fig 3: RSA private key decryption confirmed successful, showing the encrypted ciphertext decrypted using the private key and original message fully recovered.

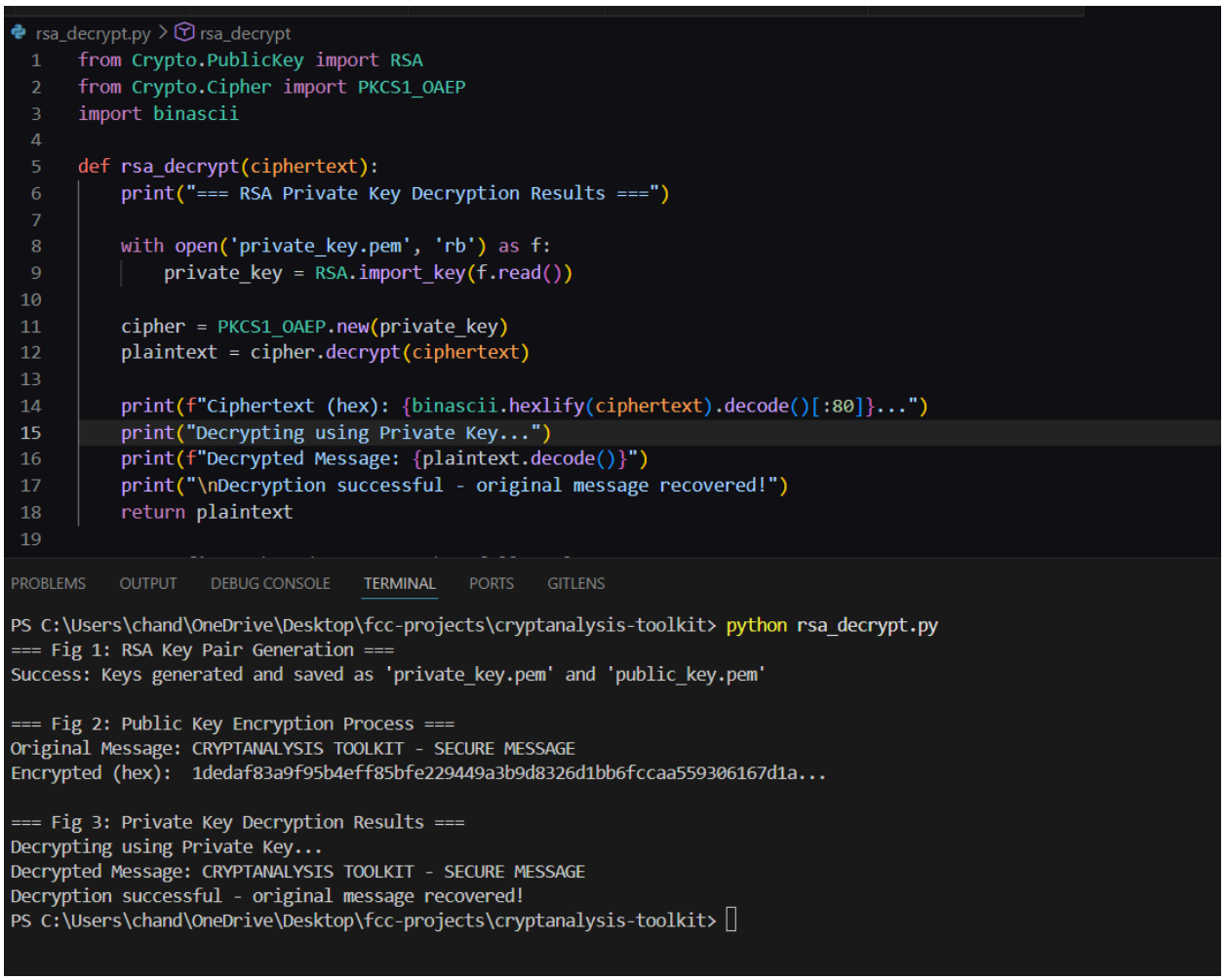


Fig 4: Secure message transmission simulated, demonstrating sender encrypting with receiver's public key and receiver successfully decrypting using their private key.

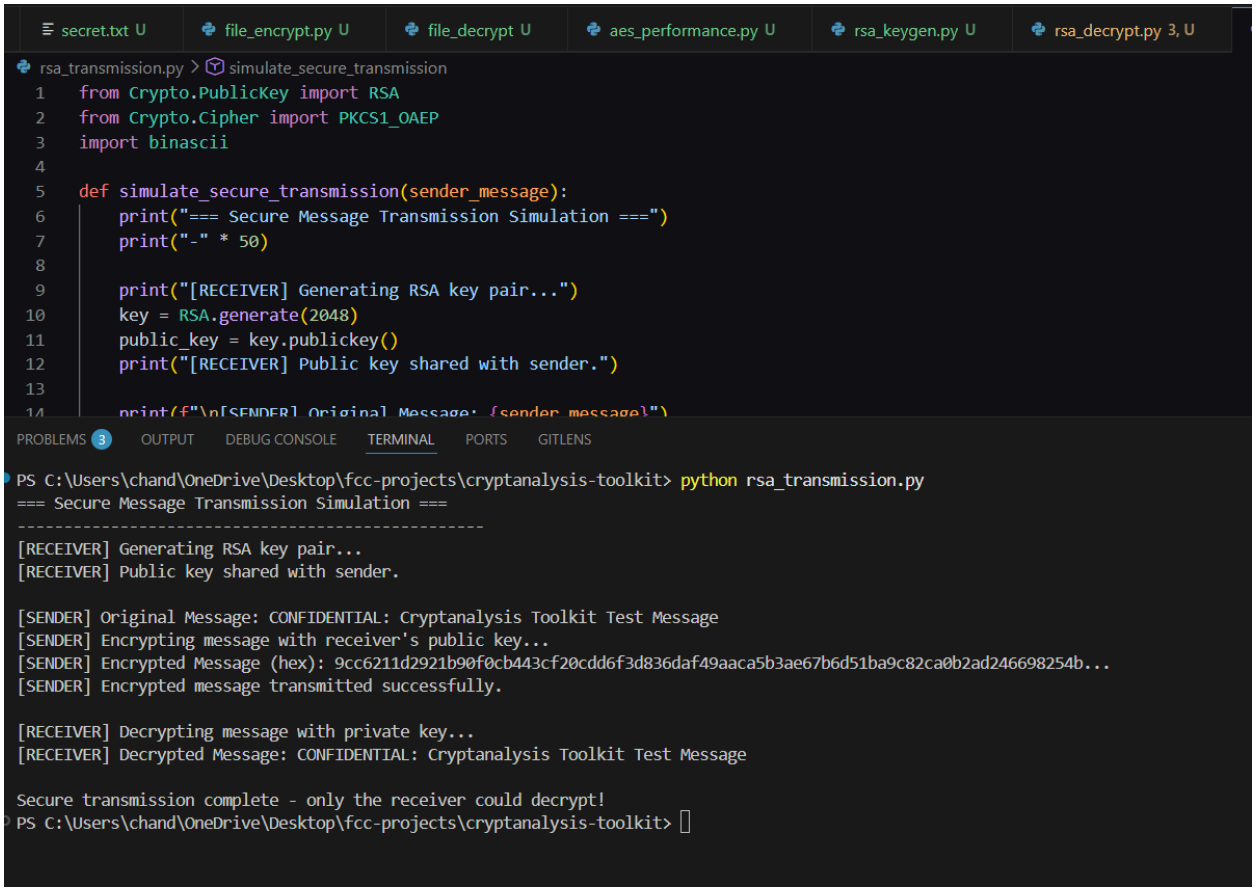
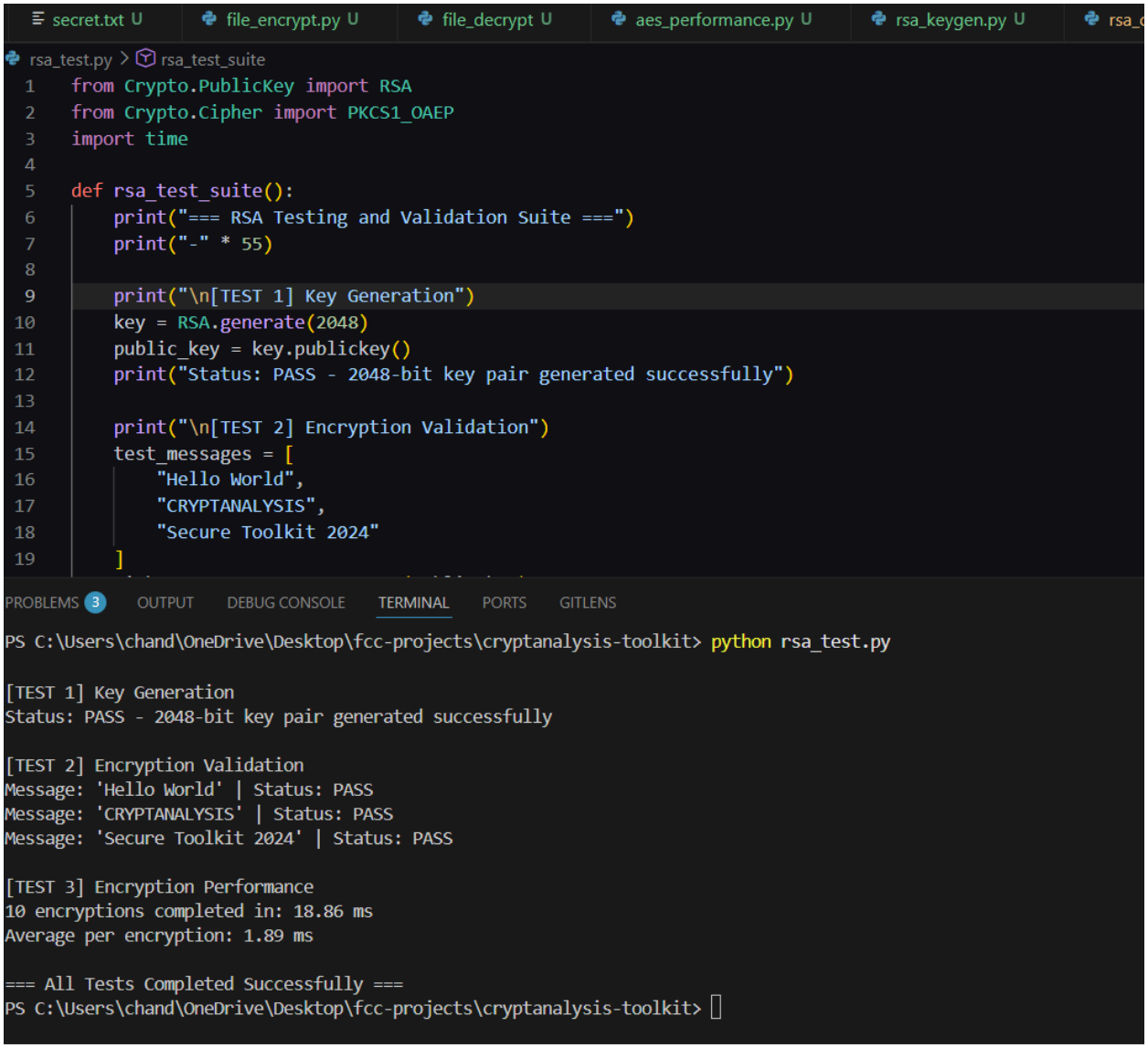


Fig 5: RSA testing and validation suite executed, with key generation, encryption validation and performance tests all returning PASS confirming a fully functional RSA implementation.



Student Reflection

Unlike symmetric encryption which uses one key for both locking and unlocking, RSA uses mathematically linked public and private keys. The public key encrypts like an open padlock, once shut only the private key can open it. RSA's security relies on prime number factorization, validated through key generation, encryption and performance tests.

Week 5 Summary

Week 5 involved implementing RSA asymmetric encryption, simulating secure message transmission between sender and receiver. A 2048-bit key pair was generated with public key encryption and private key decryption successfully demonstrated. A full validation suite confirmed correct key generation, accurate encryption and decryption, and acceptable performance across multiple test cases.

Creativity Section: Frequency Analysis Visualizer

Letter Frequency Analysis Tool — Cryptanalysis Extension Module

As an extension to the Cryptanalysis and Security Testing Toolkit, a Frequency Analysis Visualizer was implemented using Python and Matplotlib. Frequency analysis is a real-world cryptanalysis technique used to break classical ciphers by exploiting the uneven distribution of letters in natural language. This module adds an offensive analysis capability to the toolkit going beyond the standard weekly assignments.

Fig 1: Frequency Analysis Script

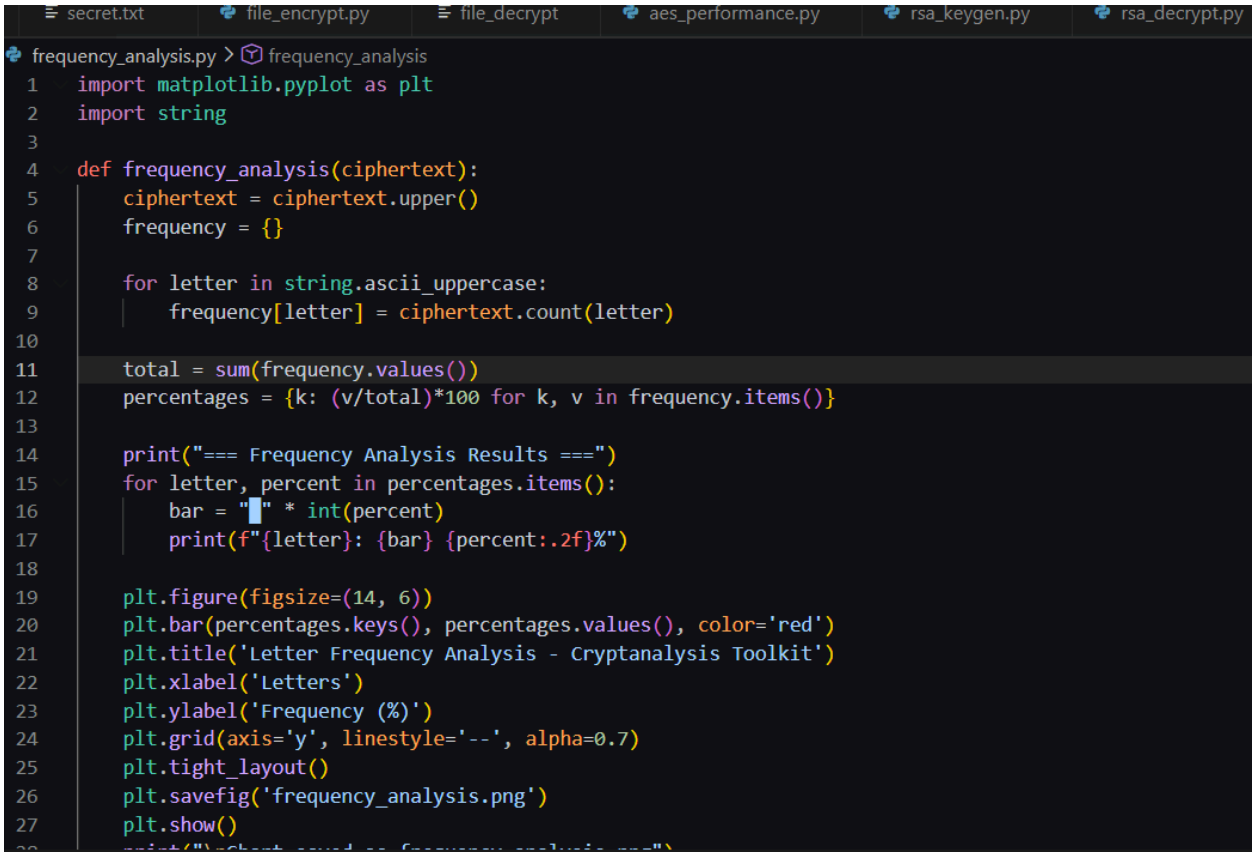


Fig 2: Terminal Frequency Output

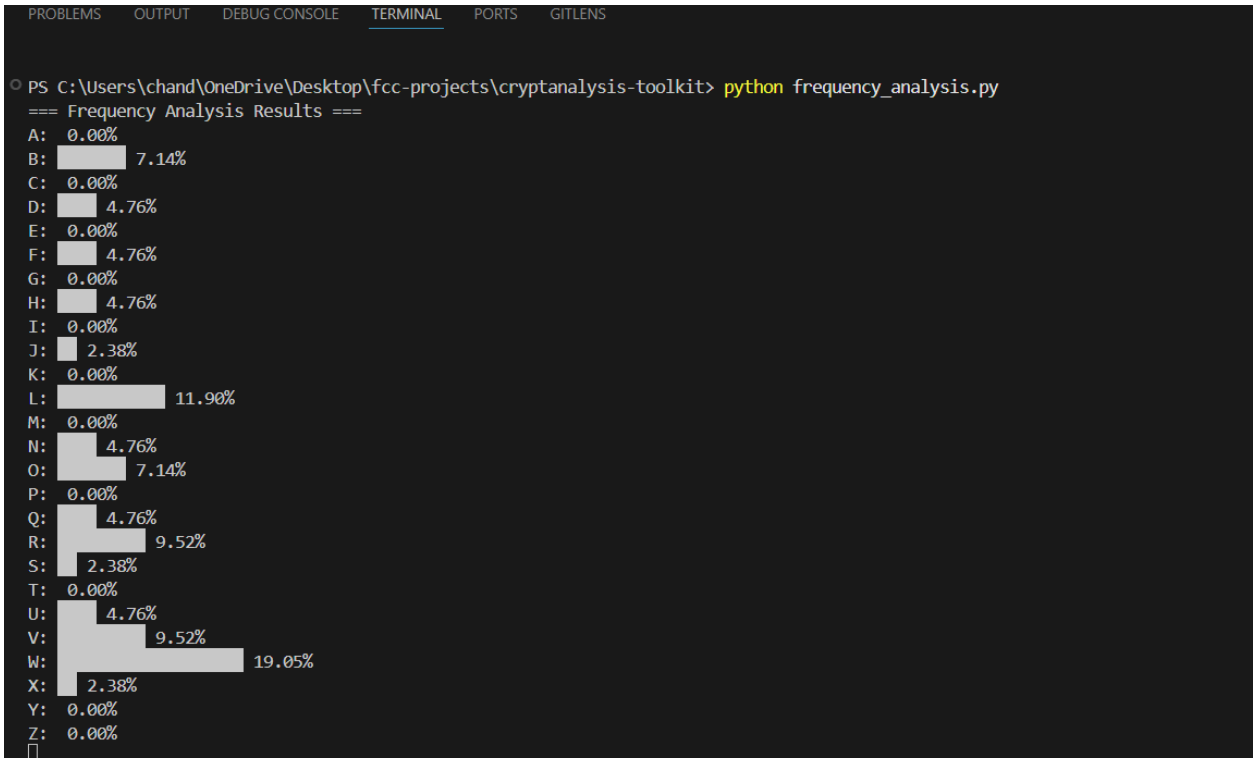
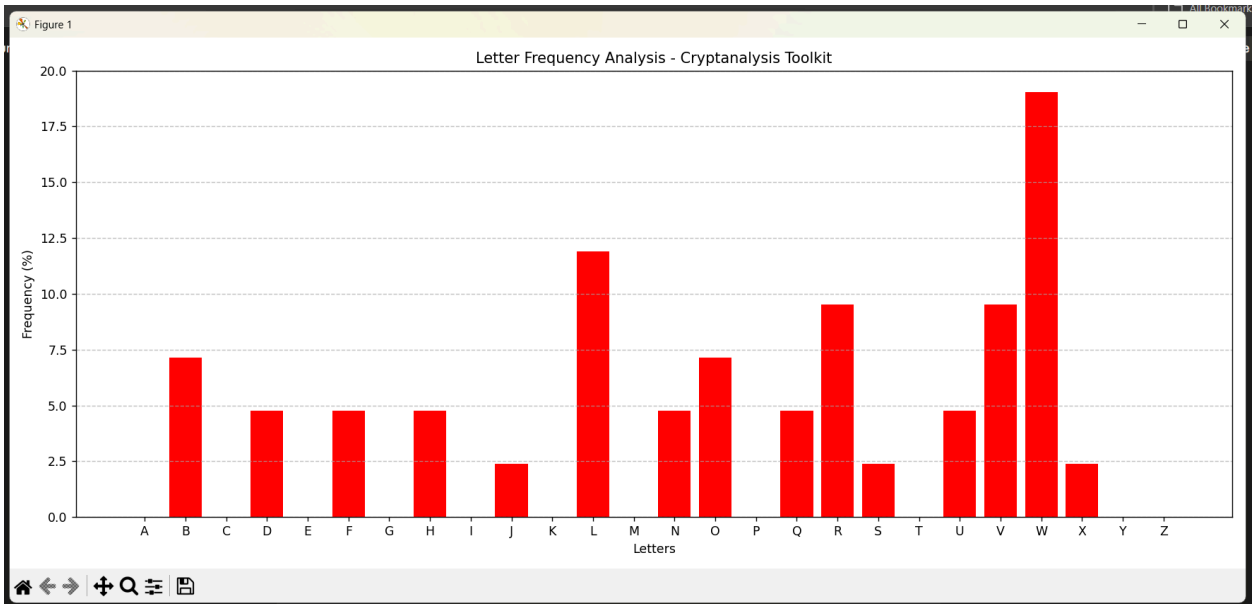


Fig 3: Matplotlib Bar Chart Visualization



Creativity Reflections

The frequency analysis visualizer extends the toolkit with a real cryptanalysis attack technique. By plotting letter distributions in ciphertext, patterns become visible that can be exploited to break classical ciphers. This demonstrates how attackers identify weaknesses in substitution ciphers, reinforcing the importance of stronger modern encryption algorithms.